

PSBT With Dilithium: Proven on BTQ Testnet

Three separate wallets co-signed real quantum-safe payments by passing PSBT files between them

Two payments were spent from a 2-of-3 quantum-safe address using standard PSBT files. Independent miners then built five blocks on top of them.

What a PSBT is, and what was missing

A **PSBT** (Partially Signed Bitcoin Transaction) is the standard file that lets several people approve one payment in turn. Think of it as a cheque that gets passed around and signed. It is how every serious multi-signature setup works: the file travels between people collecting approvals, and only becomes a valid payment once enough have signed.

BTQ already lets you lock coins behind **Dilithium** signatures, the quantum-safe scheme that replaces the maths a quantum computer could one day break. But a PSBT had no room for a quantum-safe signature. Anything Dilithium-related was discarded the moment the PSBT was saved, so a half-signed payment could not be sent on to a colleague. Everyone had to sit inside the same wallet on the same machine.

PSBT now carries Dilithium signatures. A company treasury, exchange or custodian can require two of three people to approve a payment, each on their own computer, each holding their own key.

What we proved

Three wallets, belonging to Alice, Bob and Carol, each generated one quantum-safe key and never saw the other two. All three independently produced the **same** address, and none could spend alone.

Address (2-of-3)	tb tq1zv5frdx000hfzyljdr cdl t j2z3gy36jqucf8y53pju0g5vs n76esg6s0s2
Each Dilithium signature	2,421 bytes, against roughly 71 for a normal Bitcoin signature
Resulting PSBT	About 12 KB: fine for email, far too big for a QR code

Two payments were made, deliberately using the two different ways a group can sign a PSBT. Both went through. Click either transaction below to see it on the public block explorer.

Payment 1. Alice signed, sent the PSBT to Bob, and Bob signed it. Confirmed in block **215,455**.

6fc71b6259d6f79a041e592e848e2e125940e55ad9e642e1fcdba80a2e1e0bc9

Payment 2. Alice and Carol each signed their own copy of the PSBT, and the two copies were then merged. Confirmed in block **215,469**.

8d31a6cf96ed98967f38debccfe04bc11c5b75e1f0e4ec66b975c37175632ff5

The explorer shows each payment carrying two 2,421-byte Dilithium signatures, the shared 2-of-3 script, and one **empty slot** marking the person who did not sign. That empty slot sits in a different position in each payment, because a different pair signed each time. Anyone can confirm from public data alone who approved what.

Why this counts as independent proof

We mined those two blocks ourselves, which on its own would only show our software agreeing with itself. What settles it is what happened next. A **different, unrelated miner** produced blocks 215,470, 215,471 and 215,473 to 215,475 directly on top of ours, and three independent BTQ nodes all report the same chain tip at 215,475.

Those nodes were not running our code. Had either Dilithium signature been invalid, they would have discarded the block instead of building on it. They accepted both.

What we tried to break

Attack on the PSBT	Result
Change a single byte inside a Dilithium signature	Rejected the moment the PSBT is opened, not hours later at payment time
Pay with only one of the two required approvals	Refused. No payment is produced
Have the same person sign twice to fake a second approval	Ignored. Still counted as one approval

Forgery is caught when the PSBT is *read*, so a co-signer never wastes time reviewing a file that has already been tampered with.

Verify it yourself

Both payments are permanently recorded on the public BTQ test network. Each is 8,930 bytes on the wire, yet the fee was **816 satoshis**, so quantum-safe group approval is not expensive to send. No consensus rules were changed: the same validation code that already governed these addresses accepted these payments.

Technical specification: [design/DILITHIUM_PSBT_DESIGN.md](#) . The implementation, its live-chain proof section and the re-runnable proof script accompany the implementation pull request.